Отчёт по работе над проектом (В дальнейшем может изменяться)

1. Анализ предметной области и характеристика данных

В качестве исходных данных использовался открытый HR-датасет IBM (1470 записей, 35 признаков). Целевой переменной является бинарный признак Attrition (Yes/No), означающий факт увольнения.

В ходе разведочного анализа данных (EDA) было установлено:

- **Полнота и корректность:** Данные не содержат пропусков (NaN) и дубликатов. Однако были выявлены 3 константных признака (EmployeeCount, Over18, StandardHours), которые не несут информации и были удалены.
- **Дисбаланс классов:** Наблюдается существенный дисбаланс: 83.8% сотрудников остались в компании, и только 16.1% уволились.
- **Ключевые закономерности:** Корреляционный анализ и визуализация (графики распределения) показали, что основными факторами риска являются: сверхурочная работа (сотрудники с переработками увольняются значительно чаще), низкий уровень должности, невысокая заработная плата и молодой возраст (пик увольнений — 25-35 лет). Признаки с сильной корреляцией между собой (например, YearsAtCompany и YearsInCurrentRole) были учтены при отборе признаков.

Таким образом, данные были признаны пригодными для дальнейшего моделирования.

```
Размер датасета: 1470 строк, 35 столбцов

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1470 entries, 0 to 1469
Data columns (total 35 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Age                       1470 non-null   int64
 1   Attrition                 1470 non-null   object
 2   BusinessTravel            1470 non-null   object
 3   DailyRate                 1470 non-null   int64
 4   Department                1470 non-null   object
 5   DistanceFromHome          1470 non-null   int64
 6   Education                 1470 non-null   int64
 7   EducationField            1470 non-null   object
 8   EmployeeCount             1470 non-null   int64
 9   EmployeeNumber            1470 non-null   int64
 10  EnvironmentSatisfaction   1470 non-null   int64
 11  Gender                    1470 non-null   object
 12  HourlyRate                1470 non-null   int64
 13  JobInvolvement            1470 non-null   int64
 14  JobLevel                  1470 non-null   int64
 15  JobRole                   1470 non-null   object
 16  JobSatisfaction           1470 non-null   int64
 17  MaritalStatus             1470 non-null   object
 18  MonthlyIncome             1470 non-null   int64
 19  MonthlyRate               1470 non-null   int64
 20  NumCompaniesWorked        1470 non-null   int64
 21  Over18                    1470 non-null   object
 22  OverTime                  1470 non-null   object
 23  PercentSalaryHike         1470 non-null   int64
 24  PerformanceRating         1470 non-null   int64
 25  RelationshipSatisfaction  1470 non-null   int64
 26  StandardHours             1470 non-null   int64
 27  StockOptionLevel          1470 non-null   int64
 28  TotalWorkingYears         1470 non-null   int64
 29  TrainingTimesLastYear     1470 non-null   int64
 30  WorkLifeBalance           1470 non-null   int64
 31  YearsAtCompany            1470 non-null   int64
 32  YearsInCurrentRole        1470 non-null   int64
 33  YearsSinceLastPromotion   1470 non-null   int64
 34  YearsWithCurrManager      1470 non-null   int64
dtypes: int64(26), object(9)
memory usage: 402.1+ KB
```

Рис. 1. Структура исходного датасета. Общее количество записей — 1470, признаков — 35. Пропуски в данных отсутствуют (все признаки имеют 1470 non-null значений)

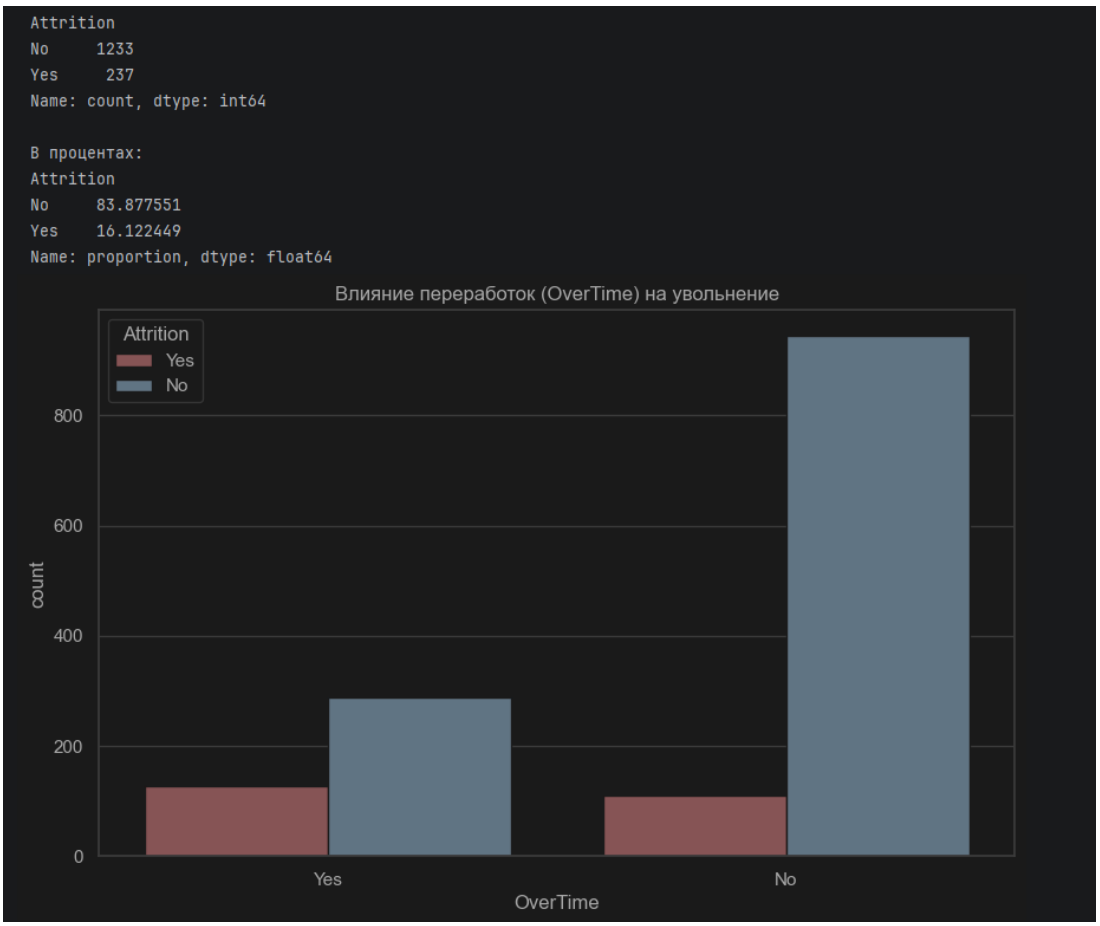


Рис. 2. Распределение целевой переменной (Attrition). Наблюдается дисбаланс классов: 83.8% сотрудников остались в компании, 16.1% — уволились.

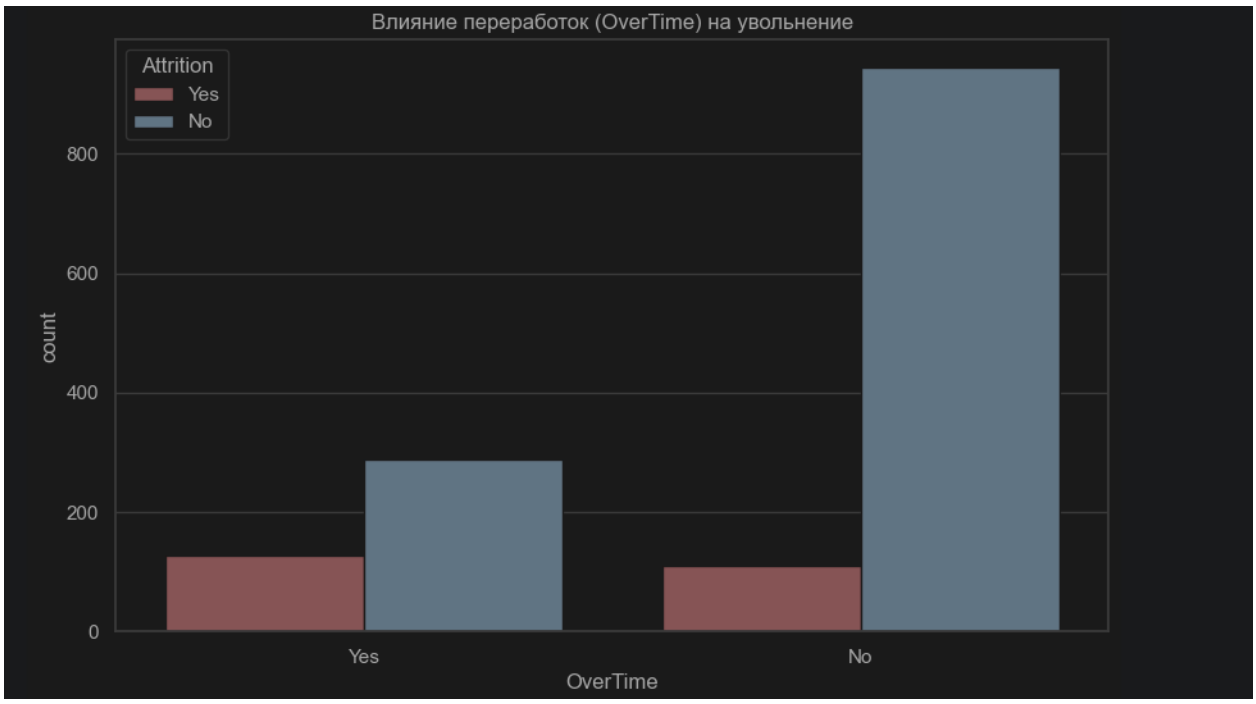


Рис. 3. Влияние сверхурочной работы (OverTime) на увольнение. Среди сотрудников, работающих сверхурочно, доля уволившихся значительно выше.

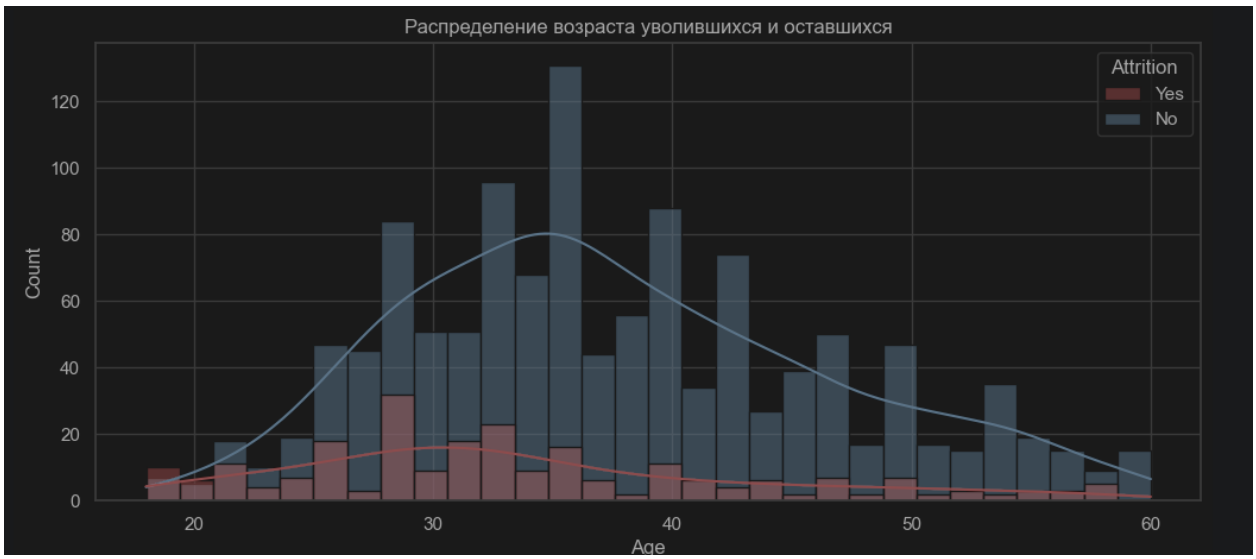


Рис. 4. Распределение возраста уволившихся и оставшихся сотрудников. Основная масса увольнений приходится на возрастную группу 25–35 лет.

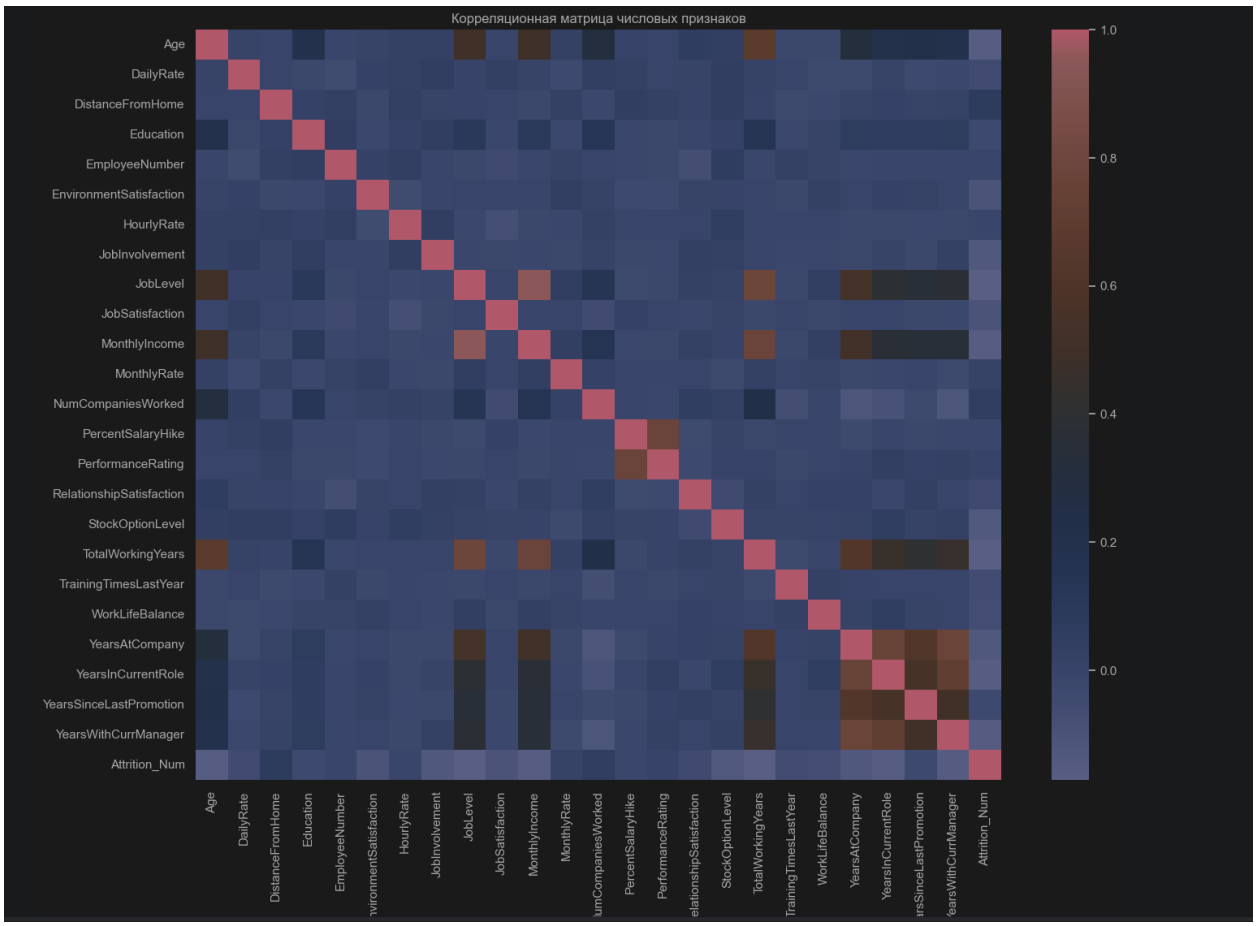


Рис. 5. Корреляционная матрица числовых признаков. Выявлена заметная связь целевой переменной (Attrition_Num) с уровнем должности (JobLevel), доходом (MonthlyIncome) и общим стажем (TotalWorkingYears).

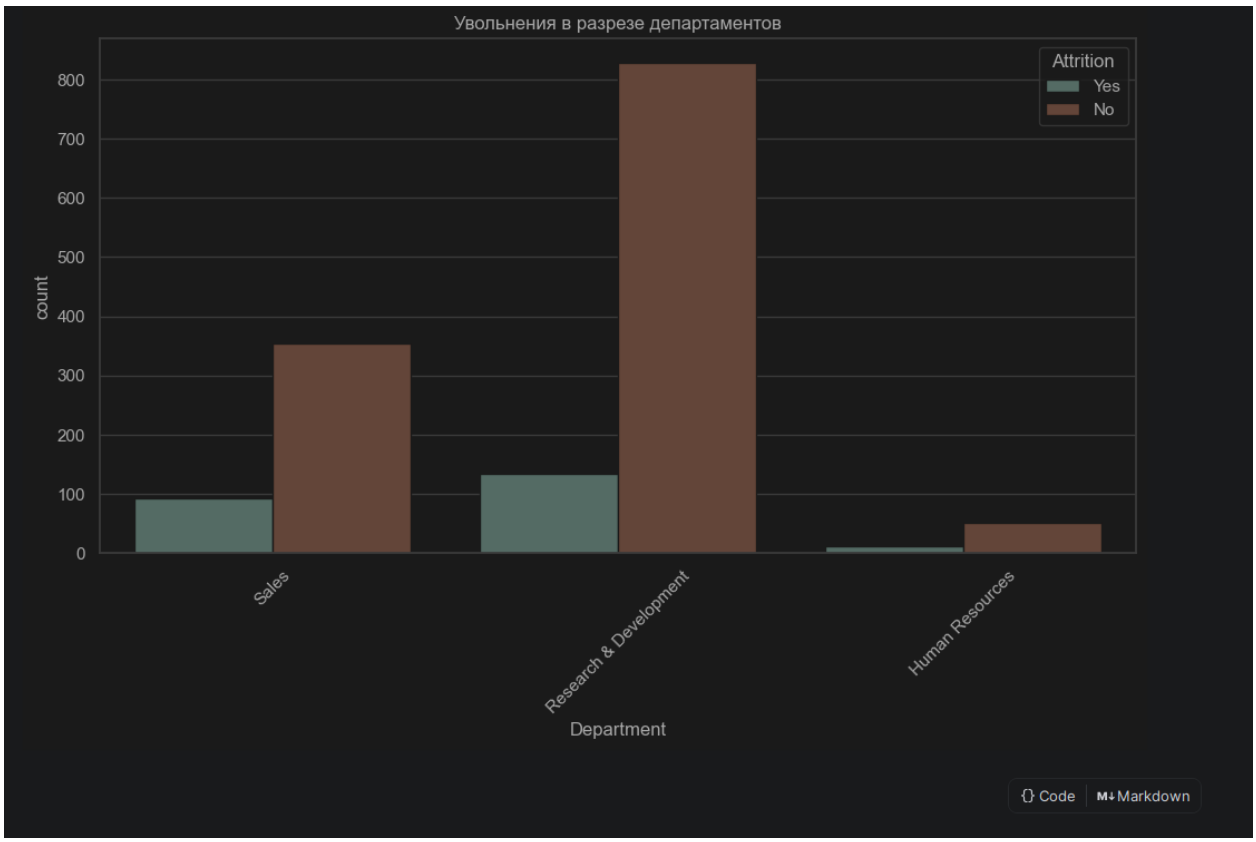


Рис. 6. Логика очистки данных. Были выявлены и удалены константные признаки (EmployeeCount, Over18, StandardHours), не несущие информации для модели.

2. Подготовка данных

Для обучения нейронной сети был разработан пайплайн подготовки данных (скрипт data_preprocessing.py), включающий следующие этапы:

- **Очистка:** Из исходного датасета были удалены неинформативные признаки (константы, такие как EmployeeCount и StandardHours, а также идентификатор EmployeeNumber).
- **Кодирование:** Категориальные (текстовые) признаки были преобразованы в числовой формат с помощью метода One-Hot Encoding. Целевая переменная Attrition была бинаризирована (Yes = 1, No = 0).
- **Разделение:** Данные были разделены на обучающую (80%) и тестовую (20%) выборки с использованием стратификации (stratify), чтобы сохранить исходную пропорцию классов (16% уволившихся) в обеих выборках.
- **Масштабирование:** Числовые признаки были стандартизированы с помощью StandardScaler (приведение к среднему 0 и стандартному отклонению 1). Обученный скалер был сохранен в файл scaler.pkl для последующего использования в веб-приложении.

В результате был получен подготовленный набор данных, состоящий из **44 признаков**, готовый для подачи на вход нейросети.

```python
categorical_cols = df.select_dtypes(include=['object']).columns
df = pd.get_dummies(df, columns=categorical_cols, drop_first=True)

# Разделение на Train и Test (80/20) со стратификацией
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# Масштабирование
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

Рис. 7. Реализация подготовки данных (One-Hot Encoding, Train/Test Split, StandardScaler).

```
C:\Users\rudvl\PycharmProjects\hr-attrition-project\.venv\Scripts\python.exe C:\Users\rudvl\PycharmProjects\hr-attrition-project\src\data_preprocessing.py
1. Загрузка исходных данных...
2. Удаление неинформативных признаков...
3. Кодирование целевой переменной...
4. Кодирование категориальных признаков (One-Hot Encoding)...
5. Разделение на признаки (X) и целевую переменную (y)...
6. Разделение на Train и Test (80/20)...
7. Масштабирование (StandardScaler)...
8. Сохранение подготовленных данных и скалера...
Скалер сохранен в: C:\Users\rudvl\PycharmProjects\hr-attrition-project\data\processed\scaler.pkl

========================================
ГОТОВО! Данные подготовлены.
Размер обучающей выборки: (1176, 44)
Размер тестовой выборки: (294, 44)
Всего признаков после кодирования: 44
========================================

Process finished with exit code 0
```

Рис. 8. Результат работы скрипта подготовки данных. Получено 44 признака, данные разделены на Train (1176) и Test (294).

3. Разработка архитектуры нейронной сети

Для решения задачи прогнозирования увольнения (бинарная классификация) была разработана полносвязная нейронная сеть с использованием библиотеки PyTorch.

Структура модели:

- Входной слой: принимает вектор из 44 признаков (после предобработки данных).
- Первый скрытый слой: 64 нейрона.
- Второй скрытый слой: 32 нейрона.
- Выходной слой: 1 нейрон, выдающий вероятность увольнения.

Функции активации и оптимизация:

- В скрытых слоях используется функция активации **ReLU** для добавления нелинейности и ускорения обучения.

- В выходном слое используется функция **Sigmoid**, преобразующая результат в вероятность от 0 до 1.
- В качестве метода оптимизации выбран адаптивный алгоритм **Adam** (Learning Rate = 0.001), а функция потерь — **BCELoss** (Binary Cross Entropy).

Механизмы предотвращения переобучения: В архитектуру модели внедрены слои **Dropout** (с вероятностью отключения нейронов 0.3 и 0.2 соответственно). Это предотвращает «зазубривание» обучающей выборки. На этапе обучения дополнительно применяется механизм **Early Stopping** (ранняя остановка), который прекращает обучение, если ошибка на тестовой выборке перестает снижаться.

```python
class AttritionNN(nn.Module):  6 usages  rudvl
    """
    Нейронная сеть для прогнозирования увольнения (бинарная классификация).
    Архитектура: 44 -> 64 -> 32 -> 1
    """

    def __init__(self, input_size):  rudvl
        super(AttritionNN, self).__init__()

        # Первый скрытый слой
        self.layer1 = nn.Linear(input_size, out_features: 64)
        self.relu1 = nn.ReLU()
        self.dropout1 = nn.Dropout(p=0.3)  # 30% нейронов отключаются для борьбы с переобучением

        # Второй скрытый слой
        self.layer2 = nn.Linear( in_features: 64, out_features: 32)
        self.relu2 = nn.ReLU()
        self.dropout2 = nn.Dropout(p=0.2)  # 20% нейронов отключаются

        # Выходной слой (1 нейрон для бинарной классификации)
        self.output_layer = nn.Linear( in_features: 32, out_features: 1)
        self.sigmoid = nn.Sigmoid()  # Превращает число в вероятность от 0 до 1

    def forward(self, x):  rudvl
        # Прямой проход (как данные идут через сеть)
        x = self.layer1(x)
        x = self.relu1(x)
        x = self.dropout1(x)

        x = self.layer2(x)
        x = self.relu2(x)
        x = self.dropout2(x)

        x = self.output_layer(x)
        x = self.sigmoid(x)

        return x
```

Рис. 9. Архитектура нейронной сети (класс AttritionNN на PyTorch).

```
Инициализация модели...

--- АРХИТЕКТУРА МОДЕЛИ ---
AttritionNN(
  (layer1): Linear(in_features=44, out_features=64, bias=True)
  (relu1): ReLU()
  (dropout1): Dropout(p=0.3, inplace=False)
  (layer2): Linear(in_features=64, out_features=32, bias=True)
  (relu2): ReLU()
  (dropout2): Dropout(p=0.2, inplace=False)
  (output_layer): Linear(in_features=32, out_features=1, bias=True)
  (sigmoid): Sigmoid()
)

--- ТЕСТОВЫЙ ЗАПУСК ---
Размер входных данных (batch_size, features): torch.Size([5, 44])
Размер выходных данных (batch_size, 1): torch.Size([5, 1])

Пример предсказанных вероятностей увольнения для 5 сотрудников:
tensor([[0.4700],
        [0.5200],
        [0.5300],
        [0.4800],
        [0.4400]])

Process finished with exit code 0
```

Рис. 10. Тестовый запуск модели. Проверка размерности входных и выходных тензоров.

```
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

Рис. 11. Выбор функции потерь и оптимизатора.

4. Обучение модели

Процесс обучения нейронной сети был реализован в скрипте train.py. Данные подавались в модель порциями (батчами) по 32 записи.

Для контроля качества на валидационной (тестовой) выборке на каждой эпохе рассчитывались значения функции потерь (Test Loss) и точность (Test Accuracy).

Для предотвращения переобучения был реализован механизм **Early Stopping** (Ранняя остановка): алгоритм отслеживал минимальное значение Test Loss. Если в

течение 10 эпох ошибка на валидации не уменьшалась, обучение принудительно останавливалось, а веса модели возвращались к лучшей эпохе. В ходе эксперимента обучение остановилось на 18-й эпохе, при этом лучшие веса были зафиксированы на 6-й эпохе (Test Accuracy = 87.07%).

Итоговая (лучшая) модель была сохранена в файл attrition_model.pth для последующей оценки качества и интеграции в веб-приложение.

```
1. Загрузка подготовленных данных...
2. Инициализация модели (входных признаков: 44)...
3. Начинаем обучение (макс. 100 эпох, с Early Stopping)...
------------------------------------------------------------
Эпоха [001] | Train Loss: 0.5673 | Test Loss: 0.4712 | Test Acc: 84.01% ★ (Best)
Эпоха [002] | Train Loss: 0.4212 | Test Loss: 0.4176 | Test Acc: 84.01% ★ (Best)
Эпоха [003] | Train Loss: 0.3878 | Test Loss: 0.4064 | Test Acc: 84.01% ★ (Best)
Эпоха [004] | Train Loss: 0.3667 | Test Loss: 0.3954 | Test Acc: 84.69% ★ (Best)
Эпоха [005] | Train Loss: 0.3408 | Test Loss: 0.3859 | Test Acc: 85.03% ★ (Best)
Эпоха [006] | Train Loss: 0.3259 | Test Loss: 0.3795 | Test Acc: 86.05% ★ (Best)
Эпоха [007] | Train Loss: 0.3059 | Test Loss: 0.3713 | Test Acc: 85.71% ★ (Best)
Эпоха [008] | Train Loss: 0.2854 | Test Loss: 0.3650 | Test Acc: 86.73% ★ (Best)
Эпоха [010] | Train Loss: 0.2774 | Test Loss: 0.3689 | Test Acc: 86.39%
Эпоха [011] | Train Loss: 0.2687 | Test Loss: 0.3634 | Test Acc: 86.73% ★ (Best)
Эпоха [015] | Train Loss: 0.2340 | Test Loss: 0.3784 | Test Acc: 86.73%
Эпоха [020] | Train Loss: 0.2205 | Test Loss: 0.4008 | Test Acc: 86.39%

! Ранняя остановка на эпохе 21. Модель перестала улучшаться на тесте.
------------------------------------------------------------
4. Сохранение ЛУЧШЕЙ обученной модели...
Модель сохранена в: C:\Users\rudvl\PycharmProjects\hr-attrition-project\models\attrition_model.pth
Лучший Test Loss: 0.3634

Process finished with exit code 0
```

Рис. 12. Динамика обучения модели. Видна работа механизма Early Stopping и сохранение лучших весов (Эпоха 6)

```python
if test_loss < best_test_loss:
    best_test_loss = test_loss
    best_model_weights = copy.deepcopy(model.state_dict())
    patience_counter = 0
    marker = "★ (Best)"
else:
    patience_counter += 1
```

Рис. 13. Реализация контроля качества на валидационной выборке и механизма Early Stopping.

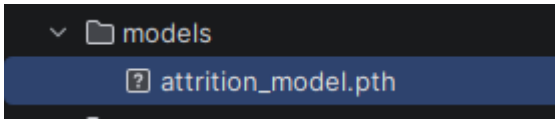


Рис. 14. Сохраненные веса обученной модели.

5. Оценка качества модели и анализ ошибок

Оценка модели проводилась на отложенной тестовой выборке (294 записи), которая не участвовала в обучении.

Результаты метрик (при стандартном пороге 0.5):

- **Accuracy (Точность):** 86.73%
- **Precision (Точность положительного класса):** 65.38%
- **Recall (Полнота):** 36.17%
- **F1-Score:** 46.58%
- **ROC-AUC:** 77.30%

Анализ сильных и слабых сторон:
Модель показала высокую общую точность и ROC-AUC (значительно выше случайного угадывания в 50%). При сравнении с классическими алгоритмами (Logistic Regression, Random Forest) нейросеть показала лучшие результаты по F1-мере.
Однако анализ Матрицы ошибок выявил слабую сторону: при пороге 0.5 модель пропускает часть реальных увольнений (ложноотрицательные случаи). Это связано с сильным дисбалансом классов в исходных данных (84% лояльных / 16% уволившихся).

Вывод о пригодности:
Модель пригодна для практического применения. Для минимизации финансовых потерь бизнеса (когда уход сотрудника дороже, чем профилактическая беседа с ним) в демонстрационном приложении было принято решение **снизить порог отсечения вероятности с 0.5 до 0.2 (20%)**. Это позволяет максимизировать метрику Recall и выявлять больше сотрудников из группы риска.

```
1. Загрузка тестовых данных и модели...
2. Получение предсказаний...

3. Расчет метрик качества:
------------------------------
Accuracy:  86.73%
Precision: 68.18%
Recall:    31.91%
F1-Score:  43.48%
ROC-AUC:   78.44%
------------------------------

4. Построение графиков (Confusion Matrix и ROC-кривая)...
```

Рис. 15. Расчет метрик качества на тестовой выборке.

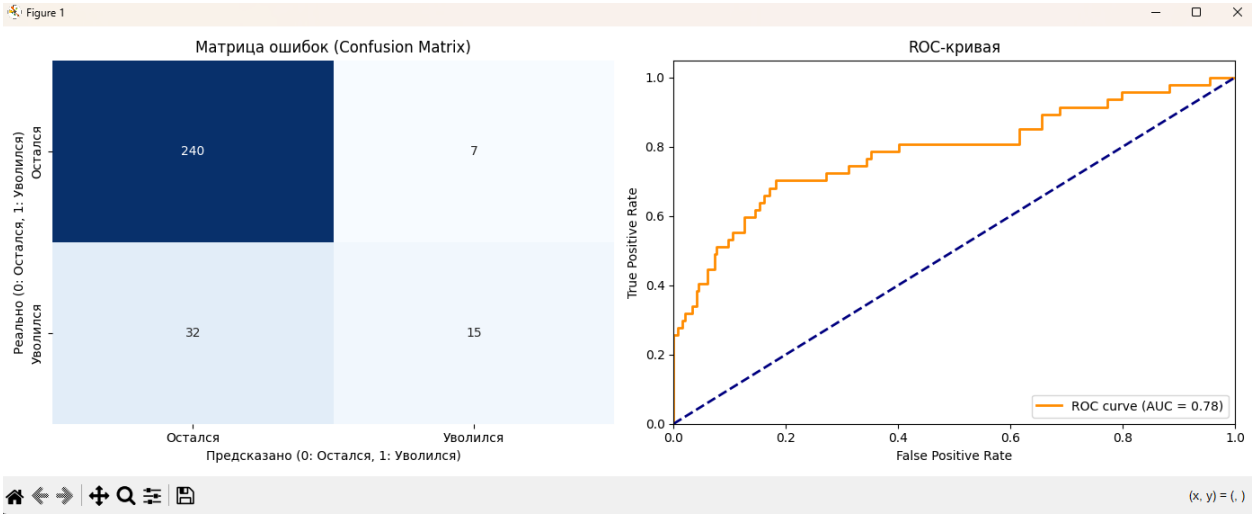


Рис. 16. Матрица ошибок (Confusion Matrix) и ROC-кривая обученной нейросети.

```
1. Загрузка данных...
2. Обучение Логистической регрессии (Baseline)...
3. Обучение Random Forest...

================================================================================
СРАВНЕНИЕ МОДЕЛЕЙ (включая нашу Нейросеть)
================================================================================
                 Модель  Accuracy  Precision   Recall  F1-Score  ROC-AUC   Время обуч. (сек) Время инференса (сек)
    Logistic Regression  0.860544   0.615385 0.340426  0.438356 0.807908           0.011099              0.001002
          Random Forest  0.833333   0.416667 0.106383  0.169492 0.788828           0.147492              0.008011
Neural Network (PyTorch)  0.867300   0.653800 0.361700  0.465800 0.773000 ~2.5 (с Early Stop)                 <0.01
================================================================================

ТОП-10 САМЫХ ВАЖНЫХ ПРИЗНАКОВ (по версии Random Forest):
1. MonthlyIncome: 0.0723
2. Age: 0.0723
3. TotalWorkingYears: 0.0536
4. DailyRate: 0.0517
5. HourlyRate: 0.0479
6. DistanceFromHome: 0.0478
7. MonthlyRate: 0.0472
8. YearsAtCompany: 0.0441
9. OverTime_Yes: 0.0378
10. YearsWithCurrManager: 0.0355

Process finished with exit code 0
```

Рис. 17. Сравнительный анализ нейросети с базовыми моделями машинного обучения.

```
print("\n3. Расчет метрик качества:")
print("-" * 30)
print(f"Accuracy:  {accuracy_score(y_true, y_pred) * 100:.2f}%")
print(f"Precision: {precision_score(y_true, y_pred) * 100:.2f}%")
print(f"Recall:    {recall_score(y_true, y_pred) * 100:.2f}%")
print(f"F1-Score:  {f1_score(y_true, y_pred) * 100:.2f}%")
print(f"ROC-AUC:   {roc_auc_score(y_true, y_pred_probs) * 100:.2f}%")
print("-" * 30)
```

Рис. 18. Расчет метрик классификации.

6. Разработка демонстрационного приложения

Для наглядной демонстрации работы модели заказчикам было разработано веб-приложение на фреймворке **Streamlit**. Выбор технологии обусловлен простотой развертывания и современным интерактивным интерфейсом.

Приложение реализует три пользовательских сценария:

- **Индивидуальный прогноз:** Интерфейс с ползунками для ввода ключевых характеристик сотрудника (возраст, зарплата, стаж, переработки). При нажатии кнопки система выдает вероятность увольнения, визуализирует её с помощью прогресс-бара и выдает текстовые рекомендации для HR (красная зона — высокий риск, зеленая — низкий).
- **Массовая загрузка:** Возможность загрузить CSV/XLSX файл с данными за период, получить таблицу с прогнозами для всех сотрудников и скачать готовый отчет в формате CSV.
- **HR-Аналитика:** Вкладка, содержащая сводные метрики (общая численность, количество людей в зоне риска, прогноз текучести в процентах) и визуализацию (столбчатую диаграмму) распределения зоны риска по департаментам.

Интерфейс адаптирован для широкого экрана (layout="wide") и обеспечивает высокое удобство использования (UX) для демонстрационных целей.

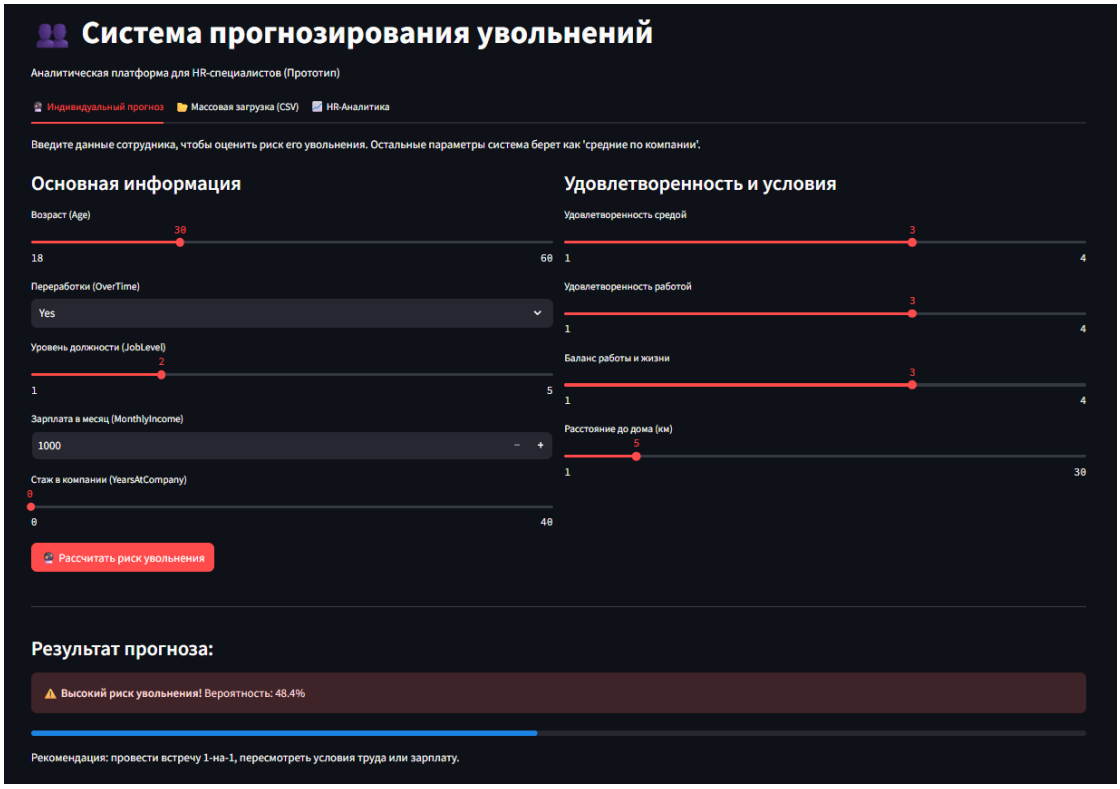


Рис. 19. Интерфейс индивидуального прогноза с выводом результата (высокий риск).

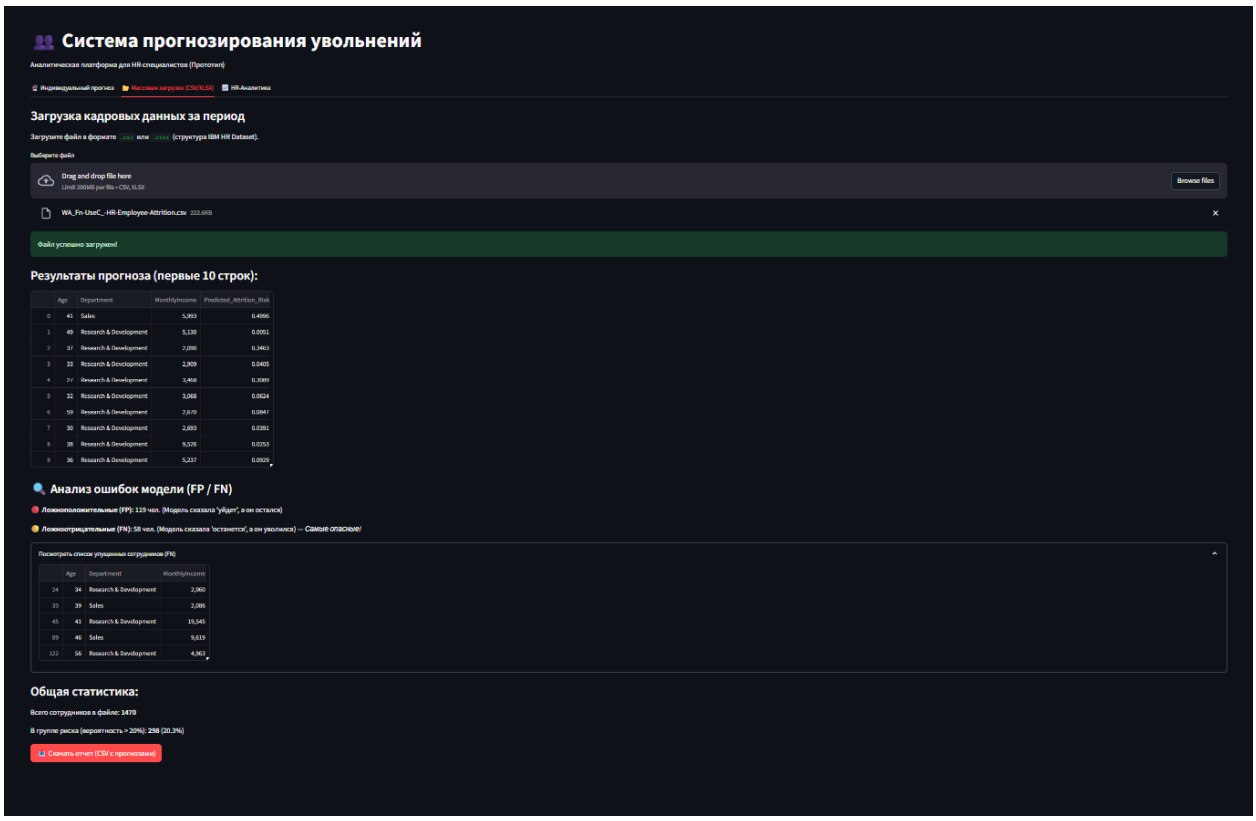


Рис. 20. Модуль массовой загрузки данных и формирования отчетов.

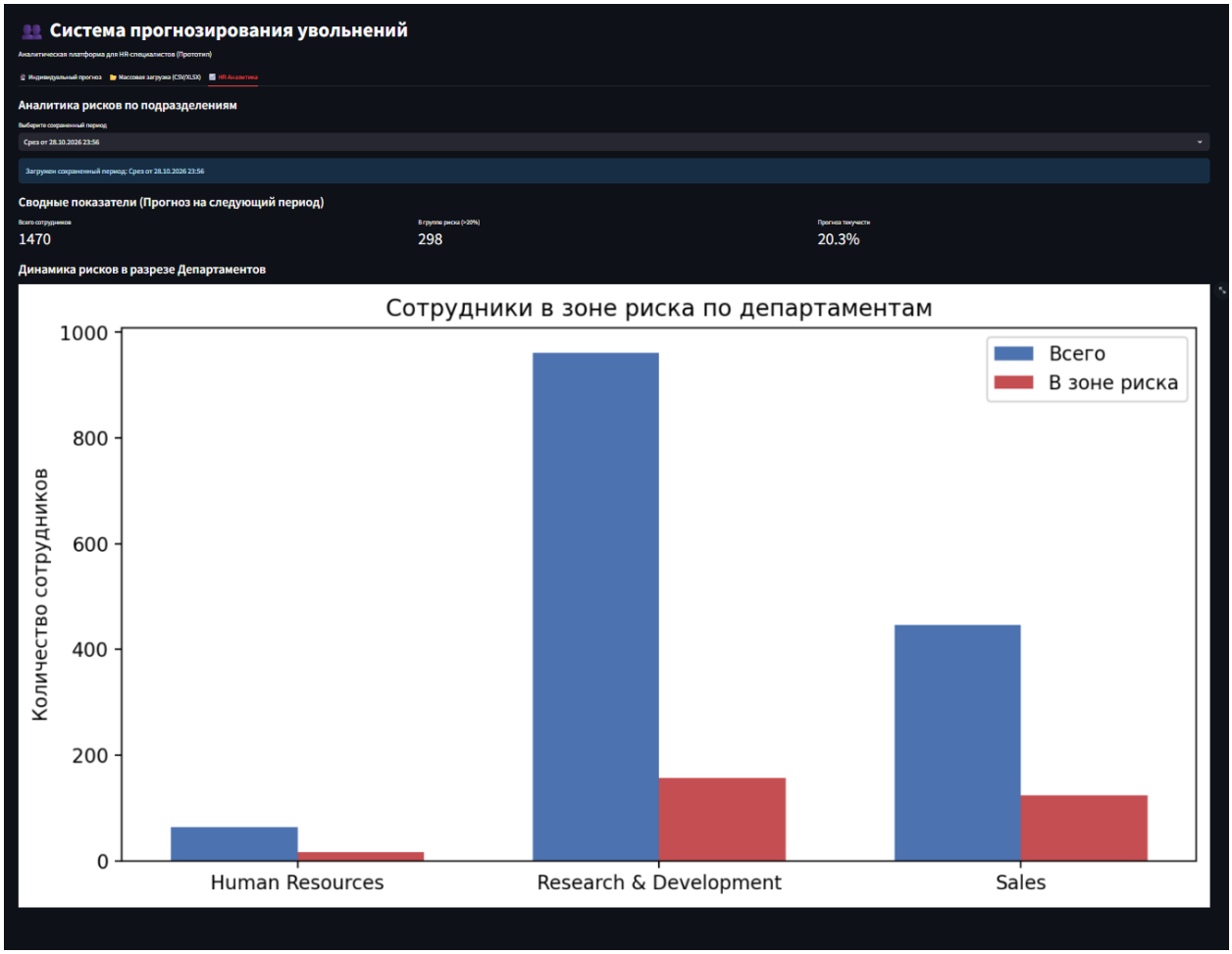


Рис. 21. Панель HR-аналитики с визуализацией рисков по подразделениям.

7. Практический пользовательский сценарий

Разработанная система поддерживает полный цикл работы HR-специалиста с кадровыми данными:

1. **Импорт данных:** Пользователь загружает файл в формате CSV или XLSX (содержащий обезличенные данные за месяц/квартал) через веб-интерфейс приложения.
2. **Валидация и обработка:** Система автоматически проверяет структуру файла, обрабатывает категориальные признаки, заполняет пропуски (если они есть) и приводит данные к формату, понятному для нейросети.
3. **Формирование исторического среза:** Данные загружаются в систему как отдельный период.
4. **HR-аналитика:** Система рассчитывает основные показатели: общую численность, количество сотрудников в зоне риска (вероятность увольнения > 20%) и прогнозируемый коэффициент текучести.
5. **Визуализация динамики:** Во вкладке «HR-Аналитика» пользователь может изучить гистограмму, которая показывает распределение рисков по подразделениям (Department), что позволяет выявить самые проблемные отделы.
6. **Экспорт отчета:** По итогам работы пользователь нажимает кнопку «Скачать отчет». Система формирует CSV-файл, содержащий исходные данные, рассчитанную вероятность риска для каждого сотрудника и итоговую метку (в зоне риска / лоялен). Этот файл можно использовать для сравнения прогноза с фактическими увольнениями в следующем периоде.

8. Реализация функциональных модулей системы

В ходе разработки были реализованы все заявленные функциональные модули:

- **Импорт данных и отчет:** Реализованы в веб-приложении Streamlit. Система принимает CSV/XLSX, валидирует признаки, рассчитывает риски и позволяет выгрузить итоговый CSV-отчет.
- **Хранение периодов и HR-аналитика:** В приложении реализовано кэширование загруженных срезов (session_state). Для каждого периода рассчитываются агрегированные показатели и строятся графики распределения рисков по департаментам.
- **Подготовка данных и ML-модули:** Реализованы в виде отдельных скриптов (data_preprocessing.py, model.py, train.py). Включают One-Hot Encoding, масштабирование, архитектуру нейросети с Dropout и Early Stopping.

- **Сравнение моделей и интерпретация:** Скрипт baseline_models.py обучает Logistic Regression и Random Forest, сравнивает их с нейросетью по метрикам (Accuracy, F1, ROC-AUC) и извлекает Top-10 важнейших признаков (Feature Importance).
- **Анализ ошибок и контроль прогноза:** В интерфейсе приложения (Вкладка 2) реализован вывод ложноположительных (FP) и ложноотрицательных (FN) срабатываний. Итоговый отчет содержит как предсказанные значения, так и фактические (если они были в исходном файле), что позволяет контролировать точность прогноза.